SYSTEM ARCHITECT AUTOMOTIVE

GIÁO TRÌNH CHUYÊN SÂU

DIAGNOSTIC RX DEMO

Luồng nhận và lắp ráp UDS Payload qua CanTp lên DCM

Biên soạn: **Phạm Văn Vũ**

HALA Academy

GIÁO TRÌNH CHUYÊN SÂU AUTOSAR DIAGNOSTIC RX (DCM)

Gắn kết CanTp, PduR và Dcm dựa trên kiến trúc chuẩn AUTOSAR

> Nếu Node TX biến khối Payload 20 Bytes thành những mẩu vụn 8-byte, thì Node RX chính là hệ thống phễu lưới, tái cấu trúc toàn diện 20 Bytes này rồi thả cho Application (Dcm) nhặt về mà không tốn một chút RAM internal overhead nào tại tầng Transport!

Bản chất hoạt động: Node RX hoàn toàn im lặng và phản ứng bằng các hàm Callbacks từ cấp thấp lên trên: `Can_Receive` -> `CanIf_RxIndication` -> `CanTp_RxIndication`. Điểm đặc sắc là PduR đóng vai trò phả lệnh cấp Buffer tĩnh từ DCM bằng việc gọi `Dcm_StartOfReception`. Tại đó dòng FC Control bắn ra để kiểm soát dòng CF dồn dập đến và mỗi CF chạm tới đều chạy hàm sao chép trực tiếp `Dcm_CopyRxData`.

Mục lục

1. Mục tiêu hệ thống Diagnostic RX
2. Những API chủ chốt được triển khai
3. Cấp phát Buffer (StartOfReception)
4. Luồng sao chép dồn (CopyRxData)
5. Sơ đồ khối và Sequence Runtime

---

1. Mục tiêu hệ thống Diagnostic RX

Phần này tập trung phân tích Node Receiver, tương dương với một ECU trong xe chờ Tester cắm vào quét lỗi UDS.

- Tìm hiểu cơ chế Buffer Cấp Phát Động của chuẩn AUTOSAR PduR. Sự tồn tại của `StartOfReception` thay vì cấp phát Hard Code buffer tại tầng Transport.
- Hiểu được tính ứng dụng của Data Length trong First Frame định hình toàn bộ luồng.
- Cách Node RX tự sinh cấu trúc Flow Control và nện vào Bus thông qua `Can_MainFunction_Write`.

> **Chuỗi đi đúng của dữ liệu trong luồng RX:**
> `Mạng Bus -> CAN HW FIFO -> Can Driver (Ngắt Rx) -> CanIf_RxIndication() -> CanTp_RxIndication(FF) -> PduR_CanTpStartOfReception -> Dcm_StartOfReception -> Sinh FC MAC -> Nhận các CF -> PduR_CanTpCopyRxData -> Dcm_CopyRxData.`

---

2. Những API chủ chốt được triển khai

Bản demo chạy `diag_rx_demo.c`. Có một dàn API được liên kết với nhau theo chiều dọc của cấu trúc.

Tên API Tại Lớp Cấp Thấp/Giữa	Quy chiếu Lớp Trực Chờ Nhận Dữ Liệu	Ý Nghĩa Khái Quát
`CanTp_RxIndication`	Chờ tại `CanTp.c`	Hứng PDU do CanIf phất cờ đưa lên. CanTp giải mã PCI Bytes. Tùy thuộc SF, FF hay CF để nhảy vào nhánh hợp lý.
`PduR_CanTpStartOfReception`	Chờ tại `PduR.c`	Khi nhận được Cờ FF lớn. CanTp không lưu giữ Data! Nó gọi PduR để xin Cấp Phát Buffer từ Lớp Application bằng cách ném Total Length lên trên. PduR phân nhánh vào thẳng `Dcm_StartOfReception`.
`PduR_CanTpCopyRxData`	Chờ tại `PduR.c`	Với mỗi CF mang lượng Payload rơi vào, CanTp bốc payload và truyền vào đây để PduR đấm tiếp xuyên thẳng lên `Dcm_CopyRxData`. Copy Data chạy trực tiếp vào Dcm Buffer không qua trung gian.

---

3. Cấp phát Buffer (StartOfReception)

3.1 Khi nhận First Frame

```
// Đoạn mã tại CanTp.c - Xử lý First Frame (FF)
uint16 ff_len = ((uint16)(pciType_byte & 0x0F) << 8) | PduInfoPtr->S

channel->rxState = CANTP_RX_WAIT_CF;
channel->rxBytesRemaining = ff_len;
channel->rxSn = 1; // Khởi động Serial Number

// Yêu cầu Dcm nhả Buffer thông qua PduR
PduLengthType bufferSizePtr = 0;
BufReq_ReturnType ret = PduR_CanTpStartOfReception(
    CanTp_Cfg_Rx[id].PduRRxPduId, // = 0 (tới DCM)
    PduInfoPtr,
    ff_len,
    &bufferSizePtr
);
```

Hành động này mang tới một sự khai phóng hoàn toàn cho CanTp. Nếu bạn nhìn kỹ mã nguồn, bạn sẽ không tìm thấy bất cứ mảng Buffer tĩnh lớn nào bên trong `CanTp.c` dành cho việc chứa chuỗi UDS 20 bytes (hoặc 4095 bytes max). Bởi vì Dcm là người chịu trách nhiệm lưu trữ!

3.2 Dcm Xin Cấp Phát

```
// Bên trong Dcm.c
BufReq_ReturnType Dcm_StartOfReception(PduIdType id, const PduInfoTy
                                       PduLengthType TpSduLength, Pd
    if (TpSduLength > DCM_RX_BUF_SIZE) return BUFREQ_E_OVFL;

    dcm_rx_length = TpSduLength; // Lưu tổng Payload
    dcm_rx_offset = 0; // Đặt ngòi nổ copy vòng lại gốc

    // Ném Size lớn nhất của Dcm xuống cho hệ thống dưới biết!
    *bufferSizePtr = DCM_RX_BUF_SIZE;

    return BUFREQ_OK;
}
```

---

4. Luồng sao chép dồn (CopyRxData)

4.1 Quá Trình Trả Flow Control CTS

Một khi DCM đã gật đầu bằng lệnh `BUFREQ_OK`, CanTp lập tức đúc ra Tấm Thẻ Flow Control (FC) và nhờ CanIf ném lại mạng CAN!

```
// CanTp.c - Phát FC
uint8 fcData[3];
fcData[0] = 0x30; // 0x3 (FC) + Lưu trạng thái CTS (0)
fcData[1] = CanTp_Cfg_Rx[id].RxBs;    // Theo config: 0 = gửi luôn r
fcData[2] = CanTp_Cfg_Rx[id].RxSTmin; // Theo config: 0ms

PduInfoType fcPdu = { .SduDataPtr = fcData, .SduLength = 3 };
CanIf_Transmit(CanTp_Cfg_Rx[id].CanIfTxFcPduId, &fcPdu); // ID 0x7E8
```

4.2 Từng Giọt Payload Lọt Vào Dcm_CopyRxData

TX Node sau khi thấy FC đã thỏa mãn gào ầm lên và rớt vào trạng thái CF Mode, dội liên hoàn đạn CF vào mặt Node RX. Trong Node RX, cứ mỗi lần chạm CF:

```
// CanTp nhận CF
uint8 sn = pciType_byte & 0x0F;
PduInfoType cfPayload;
cfPayload.SduDataPtr = &PduInfoPtr->SduDataPtr[1];
cfPayload.SduLength = PduInfoPtr->SduLength - 1;

// Gọi PduR đẩm Payload xuyên vào Dcm
PduLengthType available = 0;
PduR_CanTpCopyRxData(CanTp_Cfg_Rx[id].PduRRxPduId, &cfPayload, &avai
```

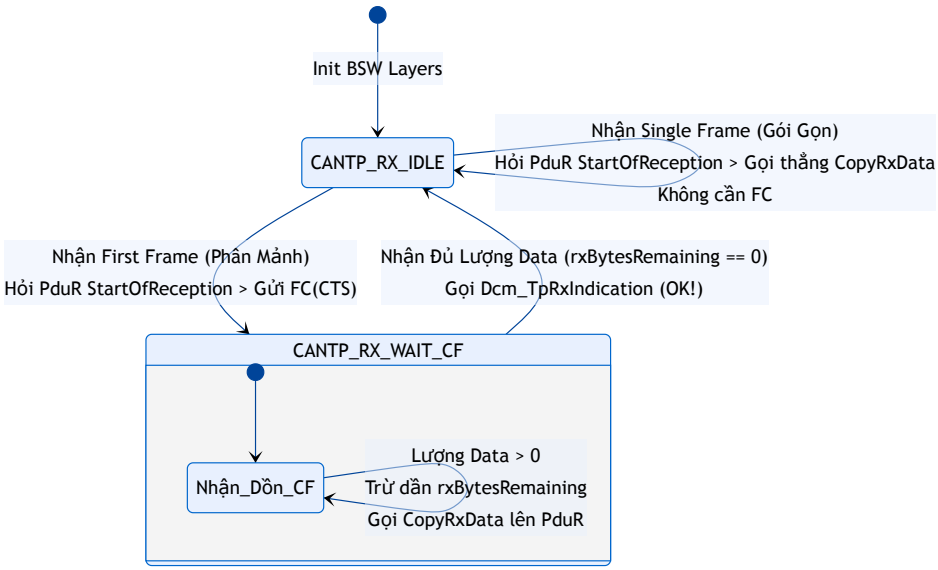
Và tại Dcm:

```
// Dcm.c
BufReq_ReturnType Dcm_CopyRxData(PduIdType id, const PduInfoType* in
    // Copy Payload Tươi vào mảng dcm_rx_buffer theo vết Index dcm_
    memcpy(&dcm_rx_buffer[dcm_rx_offset], info->SduDataPtr, info->Sd
    dcm_rx_offset += info->SduLength;
    *bufferSizePtr = DCM_RX_BUF_SIZE - dcm_rx_offset;
    return BUFREQ_OK;
}
```

---

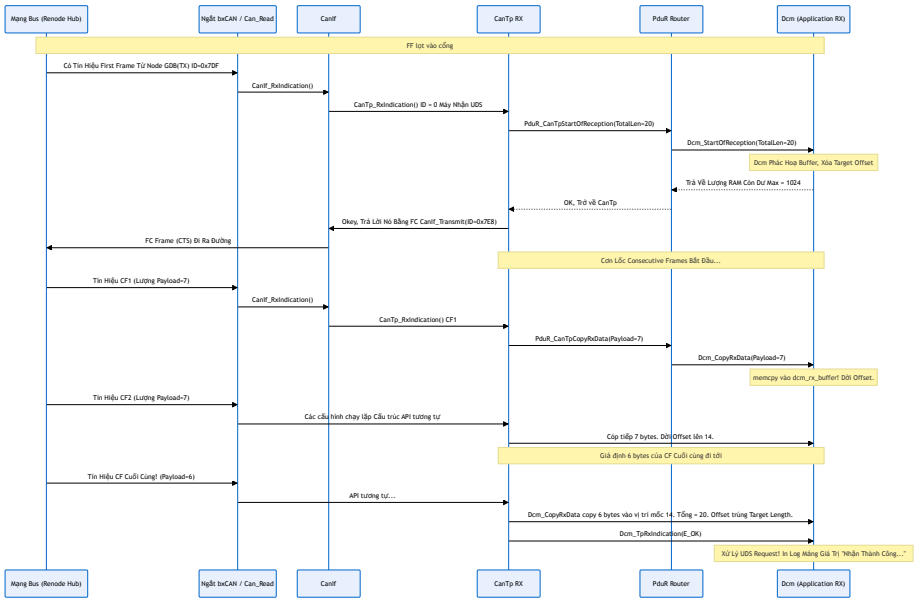
5. Sơ đồ khối và Sequence Runtime

Hình 1. CanTp State Machine - Node Nhận (RX Flow)



Hệ máy trạng thái quản lý chuỗi đời nhận tín hiệu bằng Buffer Pointer dồn Toàn Khối.

Hình 2. Sequence Diagram - Luồng Tiếp Nhận Node Dcm Test UDS



First Frame đập vào, yêu cầu DCM mở kho Buffer. Node TX gõ tiếp Consecutive Frames. RX sao chép gọn gàng cho đến byte cuối cùng. Kết quả PASS màu xanh lá bay ngập trời Terminal!